```
"""Papoine Insights Agent — an AI data-analysis agent (Streamlit + Gemini).

The agent follows the ReAct loop: Reason -> Act (call a pandas tool) -> Observe -
then writes a plain-English answer, a chart and a business recommendation.
Author: Shreya Sandeep Jagtap — Programming for Data Science, RNU
"""
import json

import matplotlib.pyplot as plt
import pandas as pd

# ======================================================================
# 1. TOOLS (plain pandas functions the agent can call)
# ======================================================================

OPS = ["==", "!=", ">", "<", ">=", "<=", "contains", "in"]


def check_column(df, col, label):
    if col not in df.columns:
        raise ValueError(f'Unknown {label} "{col}". Columns are: {", ".join(df.co


def describe_data(df, args=None):
    """Summary of the dataset: columns, types and example values."""
    cols = []
    for c in df.columns:
        s = df[c]
        missing = int(s.isna().sum())
        num = pd.to_numeric(s, errors="coerce")
        if num.notna().sum() >= 0.9 * s.notna().sum() and s.notna().any():
            cols.append({"name": c, "type": "number", "min": float(num.min()),
                         "max": float(num.max()), "mean": round(float(num.mean())
                         "missing": missing})
        else:
            counts = s.dropna().astype(str).value_counts()
            info = {"name": c, "type": "text", "distinct": int(len(counts)), "mis
            if len(counts) <= 15:
                info["values"] = sorted(counts.index.tolist())
            else:
                ordered = sorted(counts.index.tolist())
                info.update(first=ordered[0], last=ordered[-1], examples=counts.i
            cols.append(info)
    return {"row_count": int(len(df)), "columns": cols}
```

```
def _mask(series, op, value):
    if op not in OPS:
        raise ValueError(f'Unknown filter op "{op}". Use one of: {", ".join(OPS)}
    text = series.astype(str).str.lower()
    if op == "in":
        if not isinstance(value, list):
            raise ValueError('The value for "in" must be a list.')
        return text.isin([str(v).lower() for v in value])
    if op == "contains":
        return text.str.contains(str(value).lower(), regex=False)
    num = pd.to_numeric(series, errors="coerce")
    try:
        target = float(value)
        use_num = num.notna().any()
    except (TypeError, ValueError):
        use_num = False
    left, right = (num, target) if use_num else (text, str(value).lower())
    return {"==": lambda: left == right, "!=": lambda: left != right,
            ">": lambda: left > right, "<": lambda: left < right,
            ">=": lambda: left >= right, "<=": lambda: left <= right}[op]()


def apply_filters(df, filters):
    if not filters:
        return df
    if not isinstance(filters, list):
        raise ValueError('"filters" must be a list of {"column","op","value"} obj
    mask = pd.Series(True, index=df.index)
    for f in filters:
        check_column(df, f.get("column"), "filter column")
        mask &= _mask(df[f["column"]], f.get("op", "=="), f.get("value"))
    return df[mask]


def _sort_limit(s, sort, limit):
    if sort == "key":
        s = s.sort_index()
    else:
        s = s.sort_values(ascending=(sort == "asc"), na_position="last")
    try:
        limit = int(limit)
    except (TypeError, ValueError):
        limit = 15
    return s.head(max(1, min(limit, 40)))
```

```
def _rows(s):
    return [{"group": str(k), "value": None if pd.isna(v) else round(float(v), 2)
            for k, v in s.items()]


def _keys(df, group_by):
    check_column(df, group_by, "group_by column")
    col = df[group_by]
    if pd.api.types.is_float_dtype(col) and (col.dropna() % 1 == 0).all():
        col = col.astype("Int64")  # show 5 instead of 5.0
    return col.astype("string").fillna("(blank)")


def aggregate(df, args):
    """count / sum / mean / median of a column, optionally grouped and filtered."
    metric, group_by, field = args.get("metric") or "count", args.get("group_by")
    if metric not in ("count", "sum", "mean", "median"):
        raise ValueError(f'Unknown metric "{metric}". Use count, sum, mean or med
    if metric != "count":
        if not field:
            raise ValueError(f'Metric "{metric}" needs a numeric "field".')
        check_column(df, field, "field")
    data = apply_filters(df, args.get("filters"))
    if group_by:
        keys = _keys(data, group_by)
        if metric == "count":
            s = keys.value_counts()
        else:
            s = pd.to_numeric(data[field], errors="coerce").groupby(keys).agg(met
    else:
        value = len(data) if metric == "count" else getattr(
            pd.to_numeric(data[field], errors="coerce"), metric)()
        s = pd.Series({"All rows": value})
    s = _sort_limit(s, args.get("sort"), args.get("limit"))
    return {"matching_rows": int(len(data)), "rows": _rows(s)}


def percentage(df, args):
    """% of base rows that meet a condition, optionally grouped."""
    cond = args.get("condition")
    if not isinstance(cond, list) or not cond:
        raise ValueError('"condition" must be a non-empty list of filters.')
    base = apply_filters(df, args.get("filters"))
    flag = pd.Series(base.index.isin(apply_filters(base, cond).index), index=base
    if args.get("group_by"):
        s = flag.groupby(_keys(base, args["group_by"])).mean() * 100
```

```
    else:
        s = pd.Series({"All rows": flag.mean() * 100})
    s = _sort_limit(s, args.get("sort"), args.get("limit"))
    return {"base_rows": int(len(base)), "unit": "percent", "rows": _rows(s)}


TOOLS = {"describe_data": describe_data, "aggregate": aggregate, "percentage": pe
TOOL_LABELS = {"describe_data": "Checked the data structure",
               "aggregate": "Calculated totals", "percentage": "Calculated a perc

# ======================================================================
# 2. THE AGENT (instructions + ReAct loop)
# ======================================================================

SYSTEM_PROMPT = """You are the Papoine Insights Agent, a data analyst for Papoine

Reply with ONE JSON object and nothing else.

To use a tool:
{"type":"tool","thought":"<one short sentence on what you need and why>","tool":"

To finish:
{"type":"final","answer":"<2-4 plain-English sentences with the key numbers>","re

Tools:
1. describe_data - args {}. Returns columns, types and example values.
2. aggregate - args {"group_by": column or null, "metric": "count"|"sum"|"mean"|"
3. percentage - args {"condition": [filters defining the rows to count], "filters
Filter ops: "==", "!=", ">", "<", ">=", "<=", "contains", "in" (value must be a l

Rules:
- Use exact column names and values from the dataset summary.
- If a tool returns an error, read it, fix the arguments and try again.
- Use at most 4 tool calls.
- If the data cannot answer the question, say so honestly.
- Use a "line" chart for trends over time and "bar" for comparing categories. Mon

MAX_TOOL_CALLS = 4


def parse_json(text):
    clean = text.replace("```json", "").replace("```", "").strip()
    start, end = clean.find("{"), clean.rfind("}")
    return json.loads(clean[start:end + 1])


def build_prompt(question, schema, history, steps, force_final):
```

```
    steps_text = "\n\n".join(
        f"Step {i}: {s['tool']} {json.dumps(s['args'], ensure_ascii=False)}\n"
        f"Result: {json.dumps(s['result'], ensure_ascii=False)[:2500]}"
        for i, s in enumerate(steps, 1)) or "None yet."
    hist = "\n".join(f"Q: {h['q']}\nA: {h['answer']}" for h in history[-3:]) or "
    last = ("You have no tool calls left. Reply with the final JSON now."
            if force_final else "Reply with your next JSON object.")
    return (f"Dataset summary:\n{json.dumps(schema, ensure_ascii=False)}\n\n"
            f"Earlier questions in this session (for follow-ups):\n{hist}\n\n"
            f"Current question: {question}\n\nTool steps so far:\n{steps_text}\n\


def run_agent(question, df, history, llm, on_step=lambda s: None):
    """The ReAct loop. `llm` is any function: prompt text -> reply text."""
    schema, steps = describe_data(df), []
    for i in range(MAX_TOOL_CALLS + 2):
        force = len(steps) >= MAX_TOOL_CALLS or i == MAX_TOOL_CALLS + 1
        reply = llm(build_prompt(question, schema, history, steps, force))
        try:
            msg = parse_json(reply)
        except (ValueError, json.JSONDecodeError):
            on_step({"note": "The model's reply was not valid JSON. Asking again.
            continue
        if msg.get("type") == "final":
            return msg, steps
        if msg.get("type") == "tool":
            tool, args = msg.get("tool"), msg.get("args") or {}
            try:
                if tool not in TOOLS:
                    raise ValueError(f'Unknown tool "{tool}". Use: {", ".join(TOO
                result, ok = TOOLS[tool](df, args), True
            except Exception as e:  # the agent sees the error and retries
                result, ok = {"error": str(e)}, False
            step = {"thought": msg.get("thought", ""), "tool": tool, "args": args
                    "ok": ok, "result": result}
            steps.append(step)
            on_step(step)
    raise RuntimeError("The agent did not finish within its step limit. "
                       "Try a more specific question.")


def make_chart(final, steps):
    spec = final.get("chart") or {}
    idx = spec.get("step")
    if not isinstance(idx, int) or not 1 <= idx <= len(steps):
        return None
    step = steps[idx - 1]
```

```
    rows = step["result"].get("rows") if step["ok"] else None
    if not rows or len(rows) < 2:
        return None
    labels = [r["group"] for r in rows][:20]
    values = [r["value"] or 0 for r in rows][:20]
    fig, ax = plt.subplots(figsize=(7, max(3, 0.4 * len(labels) + 1)))
    if spec.get("kind") == "line":
        fig.set_size_inches(7, 3.6)
        ax.plot(labels, values, marker="o", color="#B23A6C", linewidth=2.5)
        ax.tick_params(axis="x", rotation=45)
    else:
        ax.barh(labels[::-1], values[::-1], color="#B23A6C")
    ax.set_title(spec.get("title", ""), loc="left", fontsize=12)
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(axis="x" if spec.get("kind") != "line" else "y", alpha=0.3)
    fig.tight_layout()
    return fig


# ======================================================================
# 3. WEB INTERFACE (Streamlit)
# ======================================================================

def main():
    import streamlit as st
    from google import genai
    from google.genai import types

    st.set_page_config(page_title="Papoine Insights Agent", page_icon="🌸")

    def secret(name, default=""):
        try:
            return st.secrets.get(name, default)
        except Exception:
            return default

    st.title("🌸 Papoine Insights Agent")
    st.write("Ask a question about flower orders in plain English. The agent plan
             "analysis, runs calculations on the data, checks the results, then e

    with st.sidebar:
        st.header("Settings")
        api_key = secret("GEMINI_API_KEY")
        if not api_key:
            api_key = st.text_input("Gemini API key", type="password",
                                    help="Free key from Google AI Studio")
        model = st.text_input("Model", value=secret("GEMINI_MODEL", "gemini-2.5-f
```

```
    uploaded = st.file_uploader("Upload your own CSV (optional)", type="csv")

if uploaded is not None:
    df = pd.read_csv(uploaded)
    st.info(f"Using your file: **{uploaded.name}** ({len(df)} rows)")
else:
    df = pd.read_csv("papoine_orders_sample.csv")
    st.info(f"**Sample data:** {len(df)} generated orders, {df['order_date'].
            f"{df['order_date'].max()}. Generated for this demo, not real Pap

with st.expander("Show first rows"):
    st.dataframe(df.head(10), use_container_width=True)

st.session_state.setdefault("history", [])
examples = ["Which bouquet earned the most revenue?", "Show the monthly order
            "What is the cancellation rate by plan type?",
            "Which delivery area has the highest average order value?",
            "Which weekday gets the most orders?"]
clicked = None
if uploaded is None:
    cols = st.columns(2)
    for i, ex in enumerate(examples):
        if cols[i % 2].button(ex, use_container_width=True):
            clicked = ex
typed = st.chat_input("Ask a question about the data")
question = clicked or typed
if not question:
    for h in reversed(st.session_state.history):
        st.markdown(f"**{h['q']}**  \n{h['answer']}")
    return
if not api_key:
    st.error("Add a Gemini API key in the sidebar (or in the app's Secrets) f
    return

client = genai.Client(api_key=api_key)

def llm(prompt):
    resp = client.models.generate_content(
        model=model, contents=prompt,
        config=types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT,
                                           response_mime_type="application/js
                                           temperature=0.2))
    return resp.text or ""

st.subheader(question)
with st.status("The agent is working…", expanded=True) as status:
    def show(step):
```

```
            if "note" in step:
                st.warning(step["note"])
                return
            if step["thought"]:
                st.markdown(f"*{step['thought']}*")
            if step["ok"]:
                st.markdown(f"🌿 **{TOOL_LABELS.get(step['tool'], step['tool'])}*
            else:
                st.markdown(f"🥀 **Error:** {step['result']['error']} — retrying"
            st.code(json.dumps({"tool": step["tool"], "args": step["args"]},
                               ensure_ascii=False, indent=1), language="json")
        try:
            final, steps = run_agent(question, df, st.session_state.history, llm,
            status.update(label=f"Done in {len(steps)} tool steps", state="comple
                          expanded=False)
        except Exception as e:
            status.update(label="Something went wrong", state="error")
            st.error(str(e))
            return

    st.markdown(f"### Answer\n{final.get('answer', '')}")
    fig = make_chart(final, steps)
    if fig:
        st.pyplot(fig)
    if final.get("recommendation"):
        st.success(f"**Recommendation:** {final['recommendation']}")
    st.session_state.history.append({"q": question, "answer": final.get("answer",
    st.caption("Answers come from calculations on the loaded data. "
               "Check important numbers before making decisions.")


if __name__ == "__main__":
    main()
```